Building a Real-Time Fast API Dashboard with Web Sockets and Async MySQL

This guide demonstrates how to build a real-time web application using Fast API, Web Sockets, and Async MySQL. It covers the complete setup process, including Python environment configuration, dependency installation, backend implementation, database connectivity, and running the server locally. The project shows how real-time communication can be achieved efficiently using asynchronous programming with Fast API and Web Sockets.

Requirements

- Python 3.10 or later
- MySQL Database Server
- Internet Connection
- Visual Studio Code or any Python IDE
- FastAPI
- Uvicorn
- aiomysql
- pymysql
- WebSocket support libraries

Guide Starts from Here

Step 1) Python Configuration
Check Python installed or not by entering the following 3 commands in PowerShell

```
python --version
py --version
pip --version
```

If the commands displayed the python version (python and py both are same) and pip version successfully then python is installed correctly with the pip. If they show an error, then need to download the latest version of python from the website below and install it with the instructions given below.
Website: **https://www.python.org/downloads/windows/**

Instructions to follow during the Installation

When the installer opens

1. ✓ **Tick Add Python to PATH checkbox.**
2. **Click "Customize installation"**
3. **In Installation Customizing page Tick all the checkboxes.**
4. **Then click Install.**

To Verify installation, enter the 3 commands which were used to check that python is installed or not in the start. If They displayed the python version (python and py both are same) and pip version, then python is installed correctly with the pip.

Step 2) Install Fast Api Library

```
pip install fastapi uvicorn psutil
```

Step3) Install Mysql Library

```
pip install fastapi uvicorn pymysql
```

Step4) Install Websocket Library

```
pip install 'uvicorn[standard]'
```

Step5) Upgrade the Installed Libraries and services to the latest Version

```
python.exe -m pip install --upgrade pip
```

Step6)Install aiomysql Library

```
pip install aiomysql
```

Step7) Create a python file with the name main.py and add the following code(secured).

```
import asyncio
import aiomysql  # Async MySQL library
from fastapi import FastAPI, WebSocket, WebSocketDisconnect,
Query
from fastapi.responses import HTMLResponse
from pydantic import BaseModel, ValidationError

app = FastAPI()

# -------------------------
# Async MySQL connection pool
# -------------------------
DB_CONFIG = {
    "host": "your_database_host",      # e.g., "localhost"
    "user": "your_database_username",   # e.g., "admin"
    "password": "your_secure_password", # Keep this hidden!
    "db": "your_database_name",         # database schema name
    "port": 3306                        # 3306 is the standard
}

pool = None  # will hold aiomysql pool

async def get_db_pool():
    global pool
    if pool is None:
        pool = await aiomysql.create_pool(**DB_CONFIG,
autocommit=True)
    return pool

# -------------------------
```

```
# Pydantic model for messages (example)
# -------------------------
class ClientMessage(BaseModel):
    action: str  # example field, can be extended

# -------------------------
# HTML Frontend
# -------------------------
html = """
<!DOCTYPE html>
<html>
<head>
    <title>Real-Time Database Dashboard</title>
</head>
<body>
    <h1>Live Student Records</h1>
    <div id="records"></div>

    <script>
        // Replace with wss:// in production
        const ws = new
WebSocket("ws://localhost:8000/ws?token=MY_SECURE_TOKEN");

        ws.onmessage = (event) => {
            const data = JSON.parse(event.data);
            let html = "<ul>";
            for (let row of data) {
                html += `<li>${row.SID} - ${row.Name}
(${row.Age})</li>`;
            }
            html += "</ul>";
            document.getElementById("records").innerHTML =
html;
        };
```

```
        ws.onclose = () => {
            console.log("WebSocket closed");
        };
    </script>
</body>
</html>
"""

@app.get("/")
async def get():
    return HTMLResponse(html)

# -------------------------
# WebSocket endpoint with security and efficiency
# -------------------------
@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket, token: str
= Query(...)):
    """
    token: Example JWT token or API key passed in query
params.
    In production, validate this token.
    """
    # --- Step 1: Authenticate ---
    if token != "MY_SECURE_TOKEN":  # replace with real JWT
validation
        await websocket.close(code=1008)  # Policy violation
        return

    await websocket.accept()

    try:
        db_pool = await get_db_pool()
```

```
        while True:
            # --- Step 2: Fetch data asynchronously ---
            async with db_pool.acquire() as conn:
                async with conn.cursor(aiomysql.DictCursor) as
cursor:
                    await cursor.execute("SELECT * FROM
sample")
                    rows = await cursor.fetchall()

            # --- Step 3: Send data to client ---
            await websocket.send_json(rows)

            # --- Step 4: Wait before next update ---
            await asyncio.sleep(2)

    except WebSocketDisconnect:
        print("Client disconnected")
    except Exception as e:
        print("Error:", e)
        await websocket.close(code=1011)  # Internal server
error

# -------------------------
# Run server
# -------------------------
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

*** Change the ws:// to wss:// when implementing this in a server running in the internet ***

Step8) Change the directory to the directory where the main.py python file is saved using cd command.

Step9)Start The Server

```
uvicorn main:app
```

Additional Instructions

- To Stop The Server Press CTRL + C in the Terminal
- To Reload The Server, use the command below

  ```
  uvicorn main:app
  ```
- If the file name is changed then replace the file name in start and reload server commands before running.
- Put the DB Connection Credentials before running.

GitHub Repository

Access the source code file(main.py) from the GitHub repository below:

https://github.com/NithilaLD/fastapi-websocket-realtime-dashboard